

```
Claude Code v2.1.289
Sonnet 5.5 · Claude Pro
~/idler-takehome
```

Opus 5.5 is now your default model and it draws down usage faster than Sonnet 5. Switch anytime with /model.

› Read CLAUDE.md and follow it's instructions. The repository currently contains only CLAUDE.md, so start from scratch. Before writing code, briefly tell me the implementation approach you think is best for recreating for recreating the 3D animated cube in a single self-contained HTML file.

Then create that single HTML file and establish the basic 3D rendering foundation we will build on: a full screen canvas, camera, lighting, render loop and a simple test 3D object so we can verify the rendering setup. Don't implement the cube design just yet → i'll give you the visual specification incrementally in following prompts. Run the HTML file locally and verify that it renders correctly with NO runtime errors. Make sure to keep everything in one HTML file. Don't create a framework, build system or multiple source files. Stop after the foundation is working and I'll give you the next instructions

- Approach: I'd use raw WebGL2 with a small hand-written math and shader layer, and no libraries. That keeps the file fully self-contained, with no CDN and no inlined 600KB library. It also gives full control over materials. A cube like this probably needs custom shading (edge glow, translucency, gradients), and that's easier in my own GLSL than through a library's material system. The foundation is a full-screen canvas, a perspective camera, a directional light plus ambient, a requestAnimationFrame loop, and a spinning test cube.

I'll check the environment for a way to verify it.

Ran 1 shell command (ctrl+o to expand)

- Write(index.html)
 - └ Wrote 142 lines to index.html
 - 1 <!DOCTYPE html>
 - 2 <html lang="en">
 - 3 <head>
 - 5 <meta name="viewport" content="width=device-width, initial-scale=1">
 - 6 <title>Cube</title>
 - 7 <style>
 - ound: #0b0b0f; }

```
9 canvas { display: block; width: 100vw; height: 100vh; }
10 </style>
... +150 lines (ctrl+o to expand)
```

Read 1 file, listed 1 directory, ran 1 shell command (ctrl+o to expand)

- The foundation is working, in a single file at `/home/ohmachi/idler-takehome/index.html`. I ran it in headless Chromium and it logged no console errors, warnings or page errors. I checked one screenshot, which shows the lit test cube with distinct face shading on a dark background.

I didn't check the rotation: I took a second screenshot but never looked at it.

What's in it:

- Renderer: raw WebGL2 with no libraries, so the file is self-contained.
- Canvas: full-screen, scaled for device pixel ratio (capped at 2×) and re-sized on window resize.
- Camera: perspective, 45° field of view, placed at $z=4$.
- Lighting: a directional light with ambient and diffuse terms, plus a specular highlight.
- Render loop: `requestAnimationFrame`, with depth testing and back-face culling.
- Test object: a spinning blue cube, which is placeholder geometry only.

Ready for the visual specification whenever you want to send it.

✧ Churned for 1m 2s · done 3:52 PM

✧ recap: The goal is a single-file HTML recreation of the reference cube animation, and the WebGL2 foundation (canvas, camera, lighting, render loop, spinning test cube) is built and runs without errors. Next, I'm waiting for your first visual specification. (disable recaps in `/config`)

➤ Let's start building the actual cube. The test cube is a placeholder. I'm trying to recreate a much larger cube like structure made up of lots of smaller cube pieces. It's not really a Rubik's cube. I want it to feel more like one big 3D object made out of a bunch of smaller pieces with some space between the pieces so you can tell they're separate. The overall cube should be fairly large on the screen with a white background. The camera would have that slightly angled 3 / 4 view so we can see front/top/side. I just want the basic structure right now. Replace the spinning test cube structure with this. Keep everything in the single HTML file, run it and check that it actually looks right before stopping.

- I'll replace the test cube with a 5×5×5 grid of separate small cubes with gaps between them. The camera will sit at a static 3/4 angle on a white background.

Read 1 file, ran 1 shell command (ctrl+o to expand)

- The structure reads correctly. The cube should be larger on screen, so I'll move the camera in.

Read 1 file, ran 1 shell command (ctrl+o to expand)

- That's clipped at the bottom, so I'll back the camera off a little.

Read 1 file, ran 1 shell command (ctrl+o to expand)

- The cube structure is in index.html, replacing the test cube. I ran it in headless Chromium and it logged no runtime errors. I checked the last screenshot, and the structure reads as separate pieces with visible gaps.
 - Structure: a 5×5×5 grid of 125 separate small cubes, each 0.78 wide on a 1.0 pitch, so there's a clear gap between pieces.
 - View: a static 3/4 angle that shows the front, top and right side. The cube doesn't rotate yet.
 - Look: white background, with the pieces in a flat cool gray. The light falls from the top, so the top faces are lightest and the front faces darkest.
 - Framing: the cube fills about two-thirds of the screen height. My first closer framing clipped the bottom corner, so I backed the camera off to fit.

The grid size, piece size and gap, view angle and camera distance are constants in the script, so they're easy to adjust once you describe the real proportions. Send the next part of the spec when you're ready.

✧ Worked for 43s · done 4:00 PM

- › Let's correct the basic structure before moving on. Think of the large object as 3 x 3 x 3 arrangement of larger sections. The current 5 x 5 x 5 grid is too much like a collection of separate little cubes and the gaps are too large. I want the assembled object to feel like one cohesive cube. The sections should sit very close together with only small separation btwn them so we can still tell they're separate pieces. Replace 5 x 5 x 5 structure with 3 x 3 x 3. Run it again and check results before stopping

Read 1 file, ran 1 shell command (ctrl+o to expand)

- I replaced the 5×5×5 grid with a 3×3×3 arrangement of 27 sections, and it now reads as one cohesive cube. I ran it in headless Chromium with no runtime errors and checked the screenshot.

- Spacing: each section is 0.95 wide on a 1.0 pitch, so the gaps are thin. They're still visible on every face, so the sections read as separate pieces.
- Framing: I moved the camera in from 18 to 12 so the smaller grid still fills a large part of the screen. The cube sits centred with margin on all sides, and the 3/4 view and white background are unchanged.

To make the gaps tighter or looser, change PIECE (currently 0.95). Send the next part of the spec when you're ready.

✱ Crunched for 14s · done 4:08 PM

› Let's add the detailed surface pattern to each of the 27 sections. Each section should have 49 small shapes arranged in a 7 x 7 grid on it's surface. The important part is that the 49 shapes belong to the section they're on meaning if that section later moves or rotations all 49 shapes would move with it as one piece. There should be diff kinds of sections. For EX. one section might consist of 49 circles with small empty circle in the middle (ring/donut like), another might have 49 solid triangles, another might have 49 solid circles, and another might have 49 solid squares, and 49 outlined squares with small circle in the middle. Within a single section, keep the shape type consistent meaning don't mix circles, triangles, squares together within same 7 x 7 grid. For now focus on getting the 7 x 7 pattern and the different shape types. Keep everything in the same single HTML file, run it and check results before stopping.

- I'll add the 7×7 pattern in the fragment shader. Each face carries UVs and each section is drawn with its own shape-type uniform. Because it's drawn with that section's own transform, all 49 shapes belong to the section and will move with it.

Read 1 file, ran 1 shell command (ctrl+o to expand)

- The patterns are visible. I'll zoom in to check the individual shape types.

Read 1 file, ran 1 shell command (ctrl+o to expand)

- Each of the 27 sections now carries a 7×7 grid of 49 shapes. I ran the page in headless Chromium with no runtime errors, and a zoomed screenshot shows all six shape types rendering cleanly.

- How the shapes belong to a section: the pattern is drawn on all six faces of each section, using that section's own shape type and its own transform. If a section later moves or rotates, its 49 shapes per face go with it.
- Shape types: solid circle, ring (donut), solid triangle, solid square, outlined square, and outlined square with a small circle in the middle. Each section uses only one type, assigned across the cube by $(x + 2y + 3z) \% 6$, so all six types appear and no two neighbours along an axis match.
- Look for now: the shapes are a lighter tint of the section colour, with anti-aliased edges. Shape size, outline thickness and the ring's hole size are constants in shapeSDF in the fragment shader.

The triangles all point the same way on a given face but not across faces, because each face's UVs are laid out independently. Tell me if you want a specific orientation.

* Brewed for 52s · done 4:15 PM

※ recap: We're recreating the cube animation in a single HTML file, and the $3 \times 3 \times 3$ cube now has a 7×7 shape pattern on each section, with six shape types verified in a zoomed screenshot. Next, I'm waiting for your following visual spec instruction. (disable recaps in /config)

› Let's fix the detailed surface pattern. For the diff kinds of shapes, right now theres currentlry it seems 6 differnet kinds there should only be 5 of them. Those 5 should be solid triangle, solid circle, solid square, circle with smaller empty circle inside of it and square with smaller empty circle in it. Also just because they're on the same block doesnt mean for each block and all faces the shape should be the same, each face for each block can contain a different set of the 49 shape. Remember to for the 49 shapes in each face of the block to make sure it's the same shape in the 7×7 grid meaning don't mean squares, circles etc with each other. Keep everything in single HTML file

- I'll cut the shapes down to five types and pick a shape for each face of each block, so one block's faces can differ. Within a face, all 49 shapes stay identical.

Read 1 file, ran 1 shell command (ctrl+o to expand)

- The surface pattern now uses five shape types, and each face of each block picks its own shape. I ran it in headless Chromium with no runtime errors, and a zoomed screenshot shows the five shapes rendering correctly.

- Five shape types: solid circle, circle with a smaller empty circle inside,

solid triangle, solid square, and square with a smaller empty circle inside.

The old outlined-square types are gone.

- Per face: each of the six faces on each of the 27 blocks gets its own randomly chosen shape type. In the screenshot, one block shows triangles on its front face, a different shape on its top, and squares with holes on its side.
- Within a face: all 49 cells in the 7×7 grid use the same shape, with no mixing.

The random assignment is seeded (seed = 7 in the script), so the layout is the same on every load. Change the seed to get a different layout.

Because the choice is random, two touching faces can end up with the same shape. If you'd rather neighbouring faces never match, say so and I'll constrain it.

✱ Cooked for 1m 2s · done 4:20 PM

✂ recap: We're recreating the big 3×3×3 cube of patterned sections in one HTML file, and I just reduced the surface shapes to five types, chosen per face. Next, I'm waiting for your next visual spec, or your call on whether touching faces should avoid matching shapes. (disable recaps in /config)

› Read CLAUDE.md and inspect the current index.html before editing. Keep the existing 3×3×3 cube, geometry, camera, and 7×7 surface patterns unchanged.

For this step, implement one fixed slice movement only so I can evaluate the motion. Start with the following for the movement: Start with the cube fully assembled and stationary. Select one fixed vertical 1/3 slice (9 sections). Move that slice slightly away from the cube first. Then rotate the entire slice around its correct central axis. Bring it back into alignment and pause on the finished configuration. Stop there, don't loop or make any other additional animations.

The 9 sections must behave as one rigid group. Their 7×7 patterns must stay attached and maintain their relative positions throughout the movement. Use a parent/group transform for the slice rather than independently animating the sections or patterns. Keep the camera and other 18 sections completely stationary. Use smooth easing: slow separation, faster rotation, and a gentle settle at the end. No randomness or multiple slices.

Before editing, briefly state your approach. Afterward, run the HTML locally, verify the animation plays without runtime errors, and confirm the slice stays rigid and the glyphs move with it. Stop once this one movement works.

Read 1 file (ctrl+o to expand)

- I'll implement the slice animation using a single group transform matrix applied to the right-hand vertical layer's 9 sections, leaving the other 18 sections at identity, and sequence it as hold → separate along +x → rotate 90° about the slice's x axis → settle → hold. · summary

Read 1 file, ran 1 shell command (ctrl+o to expand)

- No errors from the page. PIL isn't installed, so I'll view the frames directly.

Read 4 files (ctrl+o to expand)

- The one slice movement works and plays once. I ran it in headless Chromium and no console errors or warnings came up. I checked frames from the start, the separated stage, mid-rotation and the end.
 - Slice: the right-hand vertical layer (the 9 sections with $x = +1$), which is the side you can see in the 3/4 view.
 - Timeline: about 1s fully assembled and still, then a 1.4s slow separation outward by 0.6 units. After that comes a faster 0.8s turn of 90° about the slice's own central axis, and a 1.2s gentle ease-out back into the cube. It then stays on the finished state with no loop and nothing else moving.
 - Rigid group: the slice has one transform, which the shader applies to all 9 sections, and the other 18 get an identity matrix. The patterns are drawn in each section's own space, so they can't drift.
 - Stationary parts: the camera, light and the other 18 sections never change.
 - Frames checked:
 - Start: assembled cube, same as before.
 - Separated: the slice is cleanly offset to the right as a block.
 - Mid-rotation: the slice has turned and its sections stay tight against each other.
 - End: the slice is flush with the cube again.

At the end, the slice's outward-facing surfaces show different patterns than at the start. That's the 90° turn: the faces that were on the front and top are now on the top and back. The slice is then flush with the cube again.

✧ Worked for 44s · done 4:31 PM

- I want to fix the motion now, the slice shouldn't pull out stop rotate and move back anymore. I don't want the slice to move outward at all. It should stay centered in its original position and simply rotate as one rigid 3×3

slice around its correct central axis. The rotation should feel smooth. Everything else stays the same: all 9 sections, the pattern the other 18 sections stay stationary, the camera stays fixed and no looping or randomness

- I'll remove the separation and keep a single smooth rotation in place.

Read 3 files, ran 1 shell command (ctrl+o to expand)

- All the frames look identical, including the final one, which should differ after a 90° turn. I'll check why before reporting anything.

Searched for 1 pattern (ctrl+o to expand)

- The code is correct, so the problem is the test timing. Software rendering makes the screenshot timing unreliable. I'll drive the clock directly so each frame is at an exact time.

Read 5 files, ran 1 shell command (ctrl+o to expand)

- The slice now only rotates in place, with no pull-out. I ran it in headless Chromium and no console errors or warnings came up. My first real-time screenshots all looked identical, which was a software-rendering timing problem in my test. I re-checked with a clock I control and confirmed the motion at exact times.

- Movement: the separation is gone. The right-hand vertical slice (9 sections) stays in its original position and turns 90° about its own x axis, the line through its centre. The timeline is about 1s still, then a 1.8s ease-in-out turn, then it stops.
- Rigid group: all 9 sections still share a single group transform. The mid-turn frames show them rotating as one 3×3 block, with their patterns attached and in the same relative positions.
- Unchanged: the other 18 sections, the camera, the light, the 7×7 patterns and the geometry. There is no looping or randomness.
- Frames checked:
 - Start: assembled cube.
 - Early turn: the slice has started to tilt, still flush against the cube.
 - Mid-turn: the slice is rotated about 45° and the 3×3 block is visibly rigid.
 - Late turn: the slice is nearly square again.
 - End: it sits flush with the cube again. Its outward faces now show what was on the top and back, since they have turned 90°.

The slice doesn't collide with the 18 stationary sections, because rotating

about the x axis keeps it in its own layer.

* Churned for 3m 3s · done 4:39 PM

※ recap: We're recreating the 3×3×3 patterned cube animation in a single HTML file, and the current step, a single in-place 90° rotation of one slice, is done and verified. Next, send the following visual or motion spec so I can build on it.

› Read CLAUDE.md and inspect index.html, remember to keep the 3 x 3 x 3 geometry and 7 x 7 surfae patterns. Change the following thingss: Switch the camera to isometric perspective. Next remove all lighting and shading and render flat, unlit shapes. Give every section it's own smooth two hue gradient like example (cyan to lime, yellow, teal to green, magneta-> orange, purple to blue) with a few muted pink-brown sections. Colors are attached to these sections and move with it

Read 1 file (ctrl+o to expand)

- I'll set up a true isometric orthographic projection (45° yaw, ~35.26° pitch) with a flat-color shader—no lighting or normals—where each section has its own color pair and gradient direction applied in local space so it rotates with the section, keeping faces near-black. · summary

Ran 1 shell command (ctrl+o to expand)

- The edit script aborted before writing anything, because one of my edits changed a string that a later edit needed. I'll rerun it with that stray edit removed.

Read 2 files, ran 1 shell command (ctrl+o to expand)

- The assembled state looks right: isometric, flat and unlit. The mid-rotation frame is hard to read because the dark faces have no shading to separate them, so I'll check it against the other frames.

Read 2 files (ctrl+o to expand)

- The camera is now isometric, the shading is gone and every section has its own colour gradient. I ran it in headless Chromium with no console errors or warnings, and checked frames from before, during and after the slice turn.
- Isometric camera: the projection is orthographic at the true isometric angles (45° yaw, about 35.26° pitch). The top, left and right faces show at equal weight with no perspective taper.

- Flat, unlit: all lighting and shading code is removed, with no diffuse, specular or normals. Every pixel is a flat colour.
- Section gradients: each section has its own two-hue gradient running across the section. The palettes are cyan→lime, yellow→teal, teal→green, magenta→orange and purple→blue, plus 4 muted pink-brown sections. Each section's gradient runs in its own direction.
- Colours move with sections: each section's colours and gradient direction are tied to the section, so they rotate with the slice. In the frames, the slice's colours turn with it.
- Unchanged: the 3×3×3 geometry, the 7×7 patterns (5 shape types, one per face) and the slice rotation. In the mid-turn frames the 9 sections still turn as one rigid block.

I put the gradient on the shapes and made the section bodies near-black (uColor). Without shading, solid-colour faces would have blended into one flat shape, whereas this keeps the glyphs and gaps legible. If you meant the whole section face to carry the gradient, with the shapes in a contrasting colour, that's a small change in the fragment shader.

At the 45° point of the slice turn, the slice looks stretched along one diagonal. That comes from the isometric angle and the dark faces having no shading, not from the slice deforming.

✱ Brewed for 1m 29s · done 4:45 PM

- › There is no visible gradient like magenta -> orange, also remove the black from the faces of the cube, the only color produced should be from the shapes
- There are two problems. Each face only sees part of the gradient, and the gradient axis can lie mostly out of the face's plane, so the colour barely changes across a section. I'll fix the gradient so it spans each face, and I'll make the faces white so only the shapes carry colour.

Read 1 file, ran 1 shell command (ctrl+o to expand)

- The gradients are now visible, and the faces are white so only the shapes carry colour. I ran it in headless Chromium with no console errors or warnings, and checked the assembled frame at 2× zoom.
- Gradients: the cause was that each section's gradient axis often lay mostly out of a face's plane, so the colour barely changed across a face. Each section's gradient is now a diagonal sweep across the whole section, with random signs per section, and it reaches both end colours on every face. The frame shows cyan→lime, magenta→orange, purple→blue, yellow→teal and

teal→green sweeping across their shapes.

- No black: the section faces are white, the same as the page. The only colour on screen comes from the shapes.
- Unchanged: the 3×3×3 geometry, the 7×7 patterns and shape types, the isometric camera and the flat unlit rendering. The gradient is still tied to each section's own space, so it moves with the section when the slice turns. I didn't re-check the mid-turn frames after this change.

With white faces, the thin gaps between sections are barely visible. The sections now read mainly through their shapes' colours and the changes in pattern at section boundaries. If you want the separation clearer, I can make the faces or gaps a very light gray.

✧ Worked for 59s · done 5:52 PM

✧ recap: We're building a single-file WebGL recreation of the 3×3×3 patterned cube, and the current task, an isometric, flat, white-faced version with per-section gradient shapes, now renders cleanly. Next, I'm waiting for your feedback on how it looks, then your next spec.

➤ Read CLAUDE.md inspect index.html before editing. Keeping the 3 x 3 x 3 geometry, 7 x 7 patterns, camera, colors the same. Only the movement is changing. Right now the slice turn is hardcoded/ I want to be able to queue moves by this means which layer, which axis, which direction. After each move the cube should update so the next move picks the right sections. Playign 3 moves once, right layer, then top layer, then front layer, around 2 s eachs, short pauses btwn then stop. Run it, make sure theres no errors, check frames and stop.

Read 1 file (ctrl+o to expand)

- I'll have each section track a persistent rotation matrix M, computing its current cell as M applied to its home position, and select move slices by matching that layer's axis value. During a move, affected sections render with $\text{rotate}(\text{axis}, \text{angle}) \times M$, then bake and round the rotation into M once finished so the next move reads from the updated positions. · summary

Ran 1 shell command (ctrl+o to expand)

- No errors. The moves run at 1.0–3.0s, 3.4–5.4s and 5.8–7.8s, so I'll check the frames at the start, mid-move, in the pauses and at the end.

Read 6 files (ctrl+o to expand)

- After move 1 the right layer has changed, and mid-move 2 only the top layer is

rotating. The remaining mid-move frame for the front layer is the one I haven't looked at yet.

Read 1 file (ctrl+o to expand)

- Moves are now a queue of {axis, layer, dir}, and the three you asked for play once and stop. I ran it in headless Chromium with no console errors or warnings. I checked frames at the start, during each move, in each pause and at the end.
- Move format: enqueue('x', 1, 1) means axis, layer (-1, 0 or 1 along that axis) and direction (+1 or -1, right-hand rule about the axis). You can add any number of moves to the queue, and they play in order.
- Updating the cube: each section remembers its accumulated rotation, and its current grid cell is worked out from that. When a move starts, it picks the 9 sections whose current cell is in the chosen layer. When it finishes, the turn is saved into those 9 sections, rounded to exact 90°, so the next move picks from the updated positions. Colours and patterns are drawn in each section's own space, so they travel with it.
- Timing: moves take 2.0s each with 0.4s pauses between them, and the first starts after a 1s hold. They run at 1.0–3.0s (right layer, x = +1), 3.4–5.4s (top layer, y = +1) and 5.8–7.8s (front layer, z = +1). Each turn is 90° with the same ease-in-out as before, and the cube then holds still.
- Frames checked:
 - Start: the assembled cube is unchanged, with the same colours, patterns and camera.
 - After move 1: the right layer has turned, and the other 18 sections are untouched.
 - Mid-move 2: only the top layer is turning, as one rigid slab. At that point it's turned 45° so its sides line up with the camera, which makes it look like a flat wall.
 - After move 2: the top layer has changed.
 - Mid-move 3: the front layer is turning and now includes the sections moved by the earlier two moves. This is the check that the cube updates between moves.
 - End: the cube is settled and still, and the last two frames are identical.

I chose the direction (+1) for all three moves. If any of them turns the wrong way for what you had in mind, change its last argument to -1.

✳ Cooked for 58s · done 4:58 PM

✳ recap: We're building a single-file WebGL recreation of the 3×3×3 patterned cube, and the queued three-move sequence (right, top, front layers) now plays correctly with no errors. Next, send the following visual or motion spec, or tell me if any move should turn the other way.

› Read CLAUDE.md inspect index.html before editing. Keeping the 3 x 3 x 3 geometry, 7 x 7 patterns, camera, colors the same. Only the movement is changing. Let two or three layers turn at the same time. Layers moving together turn opposite directions. Test: left and right layers together then all three layers together. Then the version where all three layers on same axes. For this one all three turn the same direction but each layer starts at a slightly different time. Each layer still takes the same time to turn. Play once and stop. pLAY The two layer test pause then try three layer test/ Play once and stop

- Approach: moves become part of timed steps. Each step holds one or more moves with an optional start delay, and every move has the same 2s duration and its own start time. I'll keep the same per-section rotation state as before, but commit each move to it when that move finishes. That works for staggered layers and for later steps. Layers turned together must be on the same axis, because layers on different axes share sections and can't turn rigidly at the same time. So the three tests are all on one axis:

1. Left and right layers turn together in opposite directions.
2. All three layers turn together, alternating directions.
3. All three layers turn in the same direction, with staggered starts.

Ran 1 shell command (ctrl+o to expand)

- I wrote the axis as 'y', but you asked for left and right layers, which are the x layers. I'll fix that before testing.

Read 4 files, ran 1 shell command (ctrl+o to expand)

- In the mid-move frames it's hard to tell by eye whether the layers turn in opposite directions, so I'll check that numerically on a debug copy, without touching index.html.

Ran 1 shell command (ctrl+o to expand)

- Confirmed: test 1 shows two opposite-direction layers at matching progress, test 2 has three layers alternating +/-/+, and test 3 has all three moving the same direction but staggered 0.3s apart. Next I'll check the staggered frame and the end state. · summary

Read 2 files (ctrl+o to expand)

- Layers can now turn together, and the three tests play once in sequence and

stop. I ran it in headless Chromium with no console errors or warnings. I checked frames, and I also logged each move's axis, layer, direction and progress on a debug copy, which confirmed the directions and stagger.

- Test 1 (1.0s): the left ($x = -1$) and right ($x = +1$) layers turn together in opposite directions. The log showed them at the same progress with directions $+1$ and -1 .
- Test 2 (4.0s): all three x layers turn together, alternating directions ($+$, $-$, $+$). The log showed all three at the same progress.
- Test 3 (7.0s): all three layers turn in the same direction ($+$) on the same axis, starting 0.3s apart. The log showed progress of 0.45, 0.30 and 0.15 at one moment, and the mid-turn frame shows the layers at different angles. Each layer still takes 2.0s.
- Pauses and ending: the pauses between tests are 1.0s. The cube stops after the last layer finishes at 9.6s, and the end frame is still.

The tests build on each other, because each move updates the cube state for the next one. Colours, patterns, geometry and camera are unchanged.

One interpretation to confirm: I read "all three layers together" as three layers on the same axis, since layers on different axes share sections and can't turn rigidly at once. All three tests use the x axis (left, middle and right layers). Moves are now grouped into steps in the script, each with an optional per-move delay, so a different axis or layer set is a one-line change.

✱ Worked for 1m 16s · done 5:01 PM

- Test 2 (4.0s): all three x layers turn together, alternating directions ($+$, $-$, $+$). The log showed all three at the same progress. This shouldnt happen, if theres three layers moving they should move in same direction. Fix this

Read 1 file, ran 2 shell commands (ctrl+o to expand)

- Test 2 now turns all three layers in the same direction, simultaneously. I changed the middle layer's direction from -1 to $+1$ in the Test 2 step. I ran it in headless Chromium with no console errors, and a debug log of each move's progress confirmed the result.
- Test 2: all three x layers (-1 , 0 , $+1$) are at direction $+1$ and at the same progress at the same moment. The mid-turn frame at 45° shows the whole cube turning as one block, with everything aligned.
- Test 1: unchanged. The left and right layers turn in opposite directions, as you specified for two layers.

- Test 3: unchanged. All layers turn the same way with 0.3s staggered starts.

Test 2 and Test 3 now differ only in timing (simultaneous versus staggered).

✳ Sautéed for 30s · done 5:10 PM

➤ remove test 2 COMPLETELY. Don't fix it or change it. I don't want test 2 movement so remove the test cases, the movement animation logic that was added anything that only exists to support test 2. Keep test 1 and test 3 unchanged. Don't introduce any new movement behaviour or change anything that's not related to this

Ran 1 shell command (ctrl+o to expand)

● Test 2 is removed, and tests 1 and 3 are unchanged. I ran it in headless Chromium with no console errors. I logged each move's progress at six times to confirm that only the two remaining tests play.

- What I removed: only the Test 2 step and its comment. Nothing else existed purely for Test 2. The multi-move step helper and the per-move delay are both still used by the other two tests.

- Test 1: the left and right layers turn together in opposite directions, from 1.0s to 3.0s. The log showed both at progress 0.5 with directions +1 and -1.

- Test 3: all three layers turn in the same direction with 0.3s staggered starts, with no new behaviour. Since the old Test 2 slot is gone, it now begins at 4.0s, right after Test 1's 1.0s pause, instead of at 7.0s. It finishes at 6.6s, and the cube then stays still. The log showed progress of 0.6, 0.45 and 0.3 at 5.2s.

The comment above the remaining staggered step still says "test 3". I left it as is, since you asked me not to change anything unrelated. I can renumber it to "test 2" if you want.

✳ Crunched for 13s · done 5:12 PM

✳ recap: We're building a single-file WebGL recreation of the 3×3×3 patterned cube, and the current task, layered turn animations, is done: test 1 (opposite-direction pair) and the staggered same-direction test now play once and stop. Next, send the following visual spec or movement change.

➤ Read claude.md and look at index.html before you edit anything. Keep the 3 x 3 x 3 geometry, 7x7 patterns, camera and colors the same. The single layer movement from before (right layer, then top then front) either got changed or

lost when we added the multi layer stuff. Bring it back exactly how it was, don't alter the multi layer i want that one too -> keep test 1 and test 3. After retrieving it play the single layer sequence, test 1, test 3

- Before editing, I'll check the earlier single-layer parameters: right (x = +1), then top (y = +1), then front (z = +1), each +1, 2.0s each, 0.4s pauses, starting at 1.0s. I'll put them back ahead of tests 1 and 3.

Read 2 files, ran 1 shell command (ctrl+o to expand)

- The single-layer sequence is restored and now plays first, followed by test 1 and then test 3. It plays once and stops. I ran it in headless Chromium with no console errors or warnings.
- Single-layer sequence: right layer (x = +1), then top layer (y = +1), then front layer (z = +1). Each turns 90° in the + direction over 2.0s, with 0.4s pauses between moves. It starts after the 1s hold.
- Checked against the earlier version: I rendered eight frames spanning the whole sequence, from the start through each pause and mid-move to the end of the third move. Each frame is byte-identical to the frame from the earlier single-layer version at the same time. The sequence is therefore exactly as it was, and the cube state carries between moves the same way.
- Test 1: the left and right layers turn together in opposite directions. A debug log at 9.8s showed both at progress 0.5, with directions +1 and -1.
- Test 3: all three layers turn the same way, starting 0.3s apart. The log at 13.0s showed progress of 0.6, 0.45 and 0.3.
- Overall timeline: the single-layer moves run from 1.0s to 7.8s, with a 1.0s pause before test 1 (8.8–10.8s). Test 3 follows after another 1.0s pause (11.8–14.4s), and the cube then holds still.

The test 1 and test 3 steps are unchanged. The geometry, patterns, camera and colours are unchanged too. The only edit was adding the three single-layer steps ahead of them.

✱ Crunched for 37s · done 5:15 PM

- Read claude.md and look at index.html before you edit anything. Keep the 3 x 3 x 3 geometry, 7 x 7 pattern, camera and colors the same. Only movement is being altered -> we're adding a new movement. Don't modify the slice turn queue or tests that are already there just add this new move. This new move is i want the whole cube to twist around the corner that's facing the camera like a swirl. Split the pieces into 3 layers by how close they are to the corner to corner line going thru the middle : inner layer, middle layer, outer layer. the middle layer is turning the opposite way from inner and

outer. OUTER STARTS first but very slowly before middle and inner join and then they all move tgt. wach layer takes same time to turn kinda, halfway thru it should look like pinwheel. They all end turned 120 degrees (middle goes 120 the other way) so it lands back as normal cube while twist should take about 1 second. Run it make sure theres no werror, check frames.

- Approach: the swirl axis is the cube diagonal through the corner facing the camera. In cube space that's (1,1,1), because the isometric view looks along it. A 120° turn about that axis maps the 3×3×3 grid back onto itself, so the cube lands as a normal cube. A section's layer comes from its distance to that axis, computed from its current grid cell. That gives 15 inner, 6 middle and 6 outer sections. I'll add 'd' (diagonal) as an axis alongside x, y and z, and give moves an optional dur and turn. Every existing move takes the defaults, so the single-layer sequence and tests 1 and 3 behave exactly as before. The twist step goes at the end of the sequence.

Ran 1 shell command (ctrl+o to expand)

- The twist plays at 15.0–16.0s. After it, all 27 sections occupy 27 distinct grid cells, all inside the cube. The page logged no console errors. I'll look at the frames.

Read 4 files (ctrl+o to expand)

- The twist plays correctly. As a last check, I'll confirm the earlier single-layer sequence is still byte-identical to before.

Ran 1 shell command (ctrl+o to expand)

- The corner twist is added as a new step at the end of the sequence, and the page runs with no console errors or warnings. I checked frames before, during and after the twist, and logged each layer's progress. The right/top/front sequence still matches its earlier frames pixel for pixel. Tests 1 and 3 only differ from before by the end-of-timeline shift, and their motion code is unchanged.

- Axis: the cube diagonal through the corner facing the camera. A 120° turn about it maps the 3×3×3 grid back onto itself.
- Layers, by distance from that diagonal: inner (15 sections, the three on the diagonal plus the face-adjacent ones), middle (6) and outer (6, the far corners). The split isn't 9/9/9 because that's how the distances fall in a 3×3×3 grid. The layer is worked out from each section's current cell, so it still works after earlier moves.
- Timing: the twist plays at 15.0–16.0s, which is 0.6s after test 3 ends.

- The outer layer starts at 0s and eases in slowly: at 0.15s it's at progress 0.15.
- The middle and inner layers join at 0.25s with a 0.75s duration, so everything ends together at 1.0s.
- The middle layer turns -120° while the inner and outer turn $+120^\circ$.
- Halfway: at 0.35–0.55s the outer sections have swung out beyond the cube into a pinwheel while the middle layer counter-rotates, and the centre cluster stays compact.
- Landing: after the twist, all 27 sections sit in 27 distinct grid cells, all inside the cube, so it looks like a normal cube again. The colours and patterns have been rearranged. The sections haven't moved with each other relative to the whole, only the arrangement has changed.

To add this, I extended the shared move helpers without changing existing behaviour. A move can now have a dur and a turn (defaulting to 2.0s and 90°), and 'd' joins x, y and z as an axis. Every earlier move uses the defaults. The geometry, patterns, camera and colours are unchanged.

✱ Crunched for 1m 16s · done 5:27 PM

✂ recap: We're building a single-file WebGL recreation of the $3 \times 3 \times 3$ patterned cube with scripted layer movements. The corner twist is now added and verified with no errors, so the next step is for you to review it and send the next movement or visual change.

› There are things wrong with this twists, the pieces are flying out past the edge of the cube, the outline should stay a hexagone the entire time and the swirl should happen insite, nothing should poke out. Fix this without changing the colors, pattern camera or 120 degree landing. Keep inner and outter tunring the same way middle the opposite. DONT change any of the other moves run it and chck the frames in the middle of the twist to make sure nothign sticks out past the outline then sotp

- Approach: the hexagon is the cube's silhouette along the swirl axis, which is the camera's view direction. Rotating about that axis keeps each point's distance from the centre, but the hexagon is narrower at its edges than at its corners. The inner and middle layers never reach past the hexagon, but the outer layer's corners do swing out. I'll shrink only the outer layer toward the centre by exactly the amount its current angle needs to stay inside, and it returns to full size at the start and the end. To verify, I'll measure rendered pixels against the cube's silhouette hexagon instead of judging by eye. First, a baseline measurement of the current twist.
Ran 1 shell command (ctrl+o to expand)

- The baseline confirms the issue: the outer layer bulges about 33px outside the

hexagon mid-twist, then settles back at 60° and at the end, with the only warnings coming from my own debug probe rather than the page itself. Now I'll move on to implementing the fix. · summary

Ran 1 shell command (ctrl+o to expand)

- Reducing the extent to 1.5 cut the worst clipping from ~33px to 2.6px, but a small residual poke remains since rendered pieces only reach 1.475 and some middle-layer corners still slightly exceed the hexagon's incircle. I'll correct the constants and add a matching gentle fit for the middle layer. · summary

Ran 1 shell command (ctrl+o to expand)

- The worst case is down to 1.3px with zero pixels more than 2px outside. That's within the edge's anti-aliasing fringe, but I'll add a hair more margin and see if it reaches essentially zero.

Ran 1 shell command (ctrl+o to expand)

- The worst case is now 0.4px, which is anti-aliasing at the edge, with nothing more than 2px outside at any sampled time from 14.9 to 16.15s. I'll look at the mid-twist frames.

Read 3 files (ctrl+o to expand)

- Nothing unusual shows up in the mid-twist frames, and the swirl stays contained within the hexagon. Now I'll run the actual index.html (not the debug copy) to check for console errors and verify the earlier moves still behave correctly. · summary

Ran 1 shell command (ctrl+o to expand)

- I fixed the twist so nothing pokes outside the cube's hexagon outline at any point. I checked it by measuring the rendered pixels against the hexagon at 0.05s steps from 14.9s to 16.15s, not just by eye.

Results: before the fix, the outer layer's corners poked up to about 33px outside the hexagon in the middle of the twist. After the fix, the worst case is 0.4px, which is anti-aliasing at the edge. No pixel anywhere is more than 2px outside. The real index.html loads and plays with no console errors (the only warnings came from the pixel-reading probe in my debug copy).

What was wrong: the twist axis is the camera's view direction, so rotating

about it keeps each piece's distance from the centre. But the hexagon is narrower at its edges than at its corners, so the outer layer's corners swung past the edges.

The fix:

- Outer layer: it shrinks toward the centre just enough for its current angle to stay inside. It's full size at the vertices and at the start and end.
- Middle layer: it only grazed the edges, so it gets a slight dip of at most 4.5%, back to full size at the start and end.
- Inner layer: it never reached the edges, so I left it alone.
- Hexagon size: it's computed from the piece size and gap, so it follows those settings.

Kept the same: the inner and outer layers still turn one way and the middle layer the other, the timing and easing are unchanged, and the colours, patterns, camera and other moves are unchanged. I confirmed the single-layer sequence is pixel-identical to before. The twist still lands at exactly 120° , and the frame after it is identical to the previous version's.

Side effects: mid-twist the outer pieces get slightly smaller, by at most about 12%, so the outline stays inside the hexagon. Between the 60° marks, the pieces don't fill all six corners of the hexagon, so the silhouette looks a little rounded or notched. The corners can't stay filled while the layers rotate, since the pieces are rigid.

✧ Brewed for 3m 18s · done 5:37 PM

› it still looks the same to me.

also the shrinking trick isn't enough, it still looks like it's exploding. i want it to look like the original: the whole thing swirls inside the hexagon and the outline barely moves. the outer layer should barely look like it's moving until the end.

tell me exactly what file to open to see the change. run it, check the frames, then stop

- The twist is in `/home/ohmachi/idler-takehome/index.html`, but it plays last, about 15s after you open the file, so it's easy to miss. If you watched the first 15s and saw nothing, that's why. I'm also adding a shortcut so you can open it at `index.html#twist` and see only the twist. The fix itself is to keep the outer layer almost still until the end instead of rotating it through the middle of the twist. That's what made it look like an explosion.

Ran 1 shell command (ctrl+o to expand)

- The #twist shortcut works, and the twist now runs at 1.0–2.0s. Nothing is outside the hexagon, with a worst case of 0.3px. I'll look at the frames.

Read 10 files, ran 1 shell command (ctrl+o to expand)

- The hexagon outline stays sharp through the first half of the twist but rounds off inside while the outer layer swings around. I'll delay the outer layer longer to shorten that rounded stretch. · summary

Read 1 file, ran 1 shell command (ctrl+o to expand)

- The twist now keeps the outer layer back until late, and nothing goes outside the hexagon. The last part of the twist still looks looser than the first. I ran the real index.html in headless Chromium and it logged no console errors or warnings, both plain and with #twist. I measured the rendered pixels against the hexagon at 0.05s steps through the twist, and the worst case is 0.3px, with nothing more than 2px outside.

Which file to open: /home/ohmachi/idler-takehome/index.html. The twist is the last move in the sequence, so in the plain file it starts about 15s after load. If you watched the earlier part and saw nothing new, that's why. To see only the twist, open /home/ohmachi/idler-takehome/index.html#twist, where it starts after 1s. The plain file plays the full sequence as before.

What changed:

- Outer layer: it now creeps almost imperceptibly (about 7° in total) until 60% of the twist, then swings round to 120° in the last 40%. Before, it was rotating through the whole middle of the twist, and that was the "exploding" look.
- Inner and middle layers: they still swirl in opposite directions inside, joining at 0.25s. They stay well inside the outline.
- Unchanged: the 120° landing is pixel-identical to the previous version's. The colours, patterns, camera and other moves are unchanged, and so are the shrink-fit and the middle layer's dip.

What the frames show: at 1.30s and 1.45s the outline is still a full hexagon, and the inner and middle layers swirl inside it. Once the outer layer swings, at about 1.60–1.90s, the silhouette goes a little rounded and notched inside the hexagon and the outer pieces look smaller. That's the shrink-fit keeping them inside the hexagon. It never exceeds the hexagon, but it is no longer a sharp six-cornered outline during that last ~0.4s. A rigid layer can't rotate 120° and also keep its corners at the hexagon's corners, so some of this is unavoidable. If you want it tighter still, the options are to shorten the

outer swing further or to make the pieces dip less. Both would make the outer layer snap harder, so I stopped here.

✱ Cogitated for 1m 42s · done 5:41 PM

› change the movement so it loops forever in this order:

1 layer move -> 2 layer move -> 3 layer move -> 1 layer move -> 2 layer move
-> the twist -> then start over

- 1 layer move: one random layer turns (random axis, random layer, random direction)

- 2 layer move: like test 1, two layers on the same axis turn at the same time in opposite directions. pick the axis and which 2 layers randomly

- 3 layer move: like test 3, all 3 layers on one axis turn the same direction with staggered starts. pick the axis and direction randomly

- twist: the corner twist we already have

pick new random stuff every time, so it never repeats exactly. keep the same speed and short pauses between moves as before. keep the cube, patterns, camera and colors the same. make sure each move uses the cube's current state so the right pieces move every time

run it for a few loops, make sure there's no errors and every piece always lands in a proper spot, check some frames, then stop

- Approach: the timeline becomes a rolling schedule. The six-move cycle (1, 2, 3, 1, 2, twist) is generated with fresh random choices whenever the schedule runs within a few seconds of its end, and finished moves are discarded so it never grows. Which sections move is still decided from the cube's current state at the time each move runs. I'll replace the fixed sequence and drop the #twist shortcut, since the loop now includes the twist.

Ran 1 shell command (ctrl+o to expand)

- Found a bug: my two-layer helper re-rolls pick for each element in the filter, which can produce the wrong number of layers. I'll fix that before testing.
 - summary

Ran 1 shell command (ctrl+o to expand)

- Across all simulated frames, the 27 sections stayed in distinct cells with correct rotations, and the move counts (9 for layer moves, 15/6/6 for the twist) matched your sequence exactly. Now I'll examine the frames from the

actual file without probing. · summary

Read 6 files, ran 1 shell command (ctrl+o to expand)

- The cube now loops forever in your order: 1-layer, 2-layer, 3-layer, 1-layer, 2-layer, twist, then back to the start. I ran it for a few loops in headless Chromium with no console errors or warnings. Every moving group had the right sections, and every settled frame had each piece in a proper spot.
- Random choices, fresh every time: the schedule is generated a few seconds ahead with new random choices each loop, so it never repeats exactly. The cube, patterns, camera and colours are unchanged.
- 1-layer: random axis, layer (-1, 0 or 1) and direction.
- 2-layer: random axis, with two randomly chosen layers turning at once in opposite directions.
- 3-layer: random axis and direction, with all three layers turning together and starting 0.3s apart.
- Twist: the existing corner twist, including the hexagon fit, unchanged.
- Timing: each move still takes 2.0s. The pauses are 0.4s after a 1-layer move and 0.6s after a 2-layer or 3-layer move. There's a 1.0s pause after the twist, then the next loop starts.
- Right pieces move: each move picks its sections from the cube's current state when it plays, not from a fixed list. The logging showed 9 sections for every single-layer move, 9 per layer in the 2-layer and 3-layer moves, and 15, 6 and 6 for the twist's inner, middle and outer layers.
- Test run: I stepped through about 70 simulated seconds, covering more than four full loops, with a seeded random generator so the run was repeatable. The logged order matched your sequence, and the three layers in a 3-layer move all turned the same way. In the 340 settled frames, all 27 pieces were always in 27 distinct cells inside the cube and every rotation was an exact 90° multiple.
- Frames: I looked at the start, a single layer, two layers, three layers, mid-twist, and a frame at 200s. At 200s the cube was still a proper cube with all 27 pieces in place and the same patterns and colours.

Finished moves are discarded as they complete, so it can run indefinitely without slowing down. If the tab was in the background for a while, the missed moves are applied in order when it returns.

I removed the #twist shortcut I'd added last time, since the twist is now part of the loop. The fixed single-layer, test 1 and test 3 sequence is replaced by the random versions.

✱ Brewed for 2m 23s · done 5:44 PM

› Read claude.md and inspect index.html first. Keep the 3x3x3 geometry, camera, and movement unchanged. Make sure there are no visible gaps or edges between sections. The assembled cube should read as one hexagon made of fshape fields. Make the shape bigger and denser so each one fills most of its cell with tight spacing kinda like dot matrix. Can you make the shapes more solid and opaque, not thin outlines. but keep outlines squares rings only as occasional shape times. Add a bit more reds, oranges, alongside the green, teals, blues and purples. Each section keeps its own smooth gradient. Neighboring sections should share same one. Make sure cube is fully assembled and flush at rest with nothing sticking out. Run it and check frames then stop

Read 1 file (ctrl+o to expand)

- I'll adjust sections to fit flush with the pitch, use larger shapes with smaller holes, and weight shape types so rings and holed squares appear occasionally. I'll also add red/orange palettes with assignment logic ensuring no adjacent sections match, leaving the movement code untouched. · summary

Read 2 files, ran 1 shell command (ctrl+o to expand)

- The tiles are too dense and flush, so I'll shrink the solid squares slightly, then re-verify the moves and check that the twist still stays within the outline now that sections are bigger. · summary

Ran 1 shell command (ctrl+o to expand)

- Found the flagging issue at rest (13.9s): my measurement probe still uses the outdated 1.475 half-extent while the cube is now 1.5. I'll update the probe and re-measure. · summary

Ran 1 shell command (ctrl+o to expand)

- The cube aligns well at rest (0.4px worst case) but reaches up to 1.7-2px outside the hexagon during the twist now that sections are flush, so I'll tighten the twist fit slightly. · summary

Ran 1 shell command (ctrl+o to expand)

- The twist now stays within 0.9px of the outline. I'm doing the last frame checks on the final file, which are the resting cube, a mid-move frame and the twist.

Read 4 files, ran 1 shell command (ctrl+o to expand)

- The assembled cube is now flush, with no gaps or seams between sections, and it reads as one hexagon of tightly packed shapes. I ran it in headless Chromium and the page logged no console errors or warnings. I checked frames of the resting cube, a layer move, the twist and a staggered move, plus the loop invariants.
 - Flush, no gaps: sections now fill their whole cell instead of leaving the 0.05 gap. The 7×7 shape spacing stays consistent across section boundaries, so the shape field runs unbroken across the whole cube. At rest the outline sits exactly on the hexagon, with a worst case of 0.4px of edge anti-aliasing.
 - Bigger, denser, more solid shapes: circles fill about 92% of their cell, triangles and squares fill most of theirs, and the holes in the ring and holed-square types are bigger so their walls are thick, not thin outlines. The solid squares are slightly smaller than the cell, so they read as separate shapes in a dot-matrix pattern, not as tiles.
 - Occasional outlines: shape choice is weighted per face: 30% solid circle, 30% triangle, 26% solid square, 7% ring and 7% holed square. Rings and holed squares are therefore occasional, and each face still uses one shape type throughout.
 - More reds and oranges: I added red→orange, orange→amber, crimson→orange and blue→cyan gradients to the existing cyan→lime, yellow→teal, teal→green, magenta→orange and purple→blue. Each section keeps its own smooth gradient, and the 3 muted pink-brown sections remain.
 - Neighbouring sections: I read "should share same one" as "should not share the same one", since each section is meant to have its own gradient. Sections that touch always get different palettes. If you meant the opposite, tell me.
 - Movement: the moves, the camera and the random loop code are unchanged. Over about 45 simulated seconds, every settled frame still had all 27 pieces in distinct cells with exact rotations, and each move still moved the right number of sections. Because the sections are now bigger, I tightened the twist's fit slightly, so its worst case is 0.9px at the outline.

Mid-move, you'll see some pieces briefly overlapping each other in the twist, because the layers counter-rotate through the same space. That was already the case and is unchanged.

* Cooked for 3m 9s · done 5:50 PM

› R

read CLAUDE.md and inspect index.html first. Keep 3x3x3, 7x7 patterns, camera, and colors. Fix two things: The twist is broken. The cube isn't flush at rest, with some sections sticking out. Find the cause before editing and tell

me what it is. Likely suspects: the slice is rotating around the wrong pivot or the sections' positions rotate but their orientations don't. Fix it so a layer turn rotates the 9 sections as one rigid block around the layer's center axis, then snaps every section to exact grid positions and orientations at the end. Selecting the next layer must use the sections' updated positions.

Next, remove the white grid lines/borders around shapes so each shape floats in its cell with a small gap, no seams between sections.

Then test assembled state must look like one clean hexagon. Then play one layer turn and check frames at start, 50%, and end. The end frame must be flush again. Add a check that every section sits on an integer grid position after the move. Run it, confirm no errors, then stop.

- Before editing anything, I need to find the real cause. My earlier tests drove the clock in fixed steps, and that may hide a bug that only shows in real time, where frame gaps are uneven. So I'll run the actual page on the real clock, measure every frame, and also check that each section's rotation is a proper one, not just its position.

Ran 1 shell command (ctrl+o to expand)

- I've ruled out drift or misalignment in the state/pivot logic—positions and orientations stay perfectly on-grid throughout. The real issue is the outer layer's shrink-fit margin applying even at zero rotation, causing a visible size jump at twist start/end; I'll verify this numerically before fixing it.
- summary

Ran 1 shell command (ctrl+o to expand)

- Confirmed the outer layer snaps to 98% scale at the start and end. I'll smooth the margin fade to exactly 1.0 at 0°, 60°, and 120°, shrink the shapes for a clear floating gap, and add a grid check after each move.
- summary

Read 3 files, ran 1 shell command (ctrl+o to expand)

- Frames and checks look right for the layer turn. I changed the twist's fit margin, so I'll re-measure that the twist still stays inside the outline.

Ran 1 shell command (ctrl+o to expand)

- Cause: I couldn't reproduce sections sticking out at rest, and neither of your suspected causes is present. In a 45-second real-time run of the final file

(1344 samples, 319 of them at rest, covering every move type including the twist), every resting section sat exactly on an integer grid cell. Each orientation was a proper rotation (the checked entries are exactly 0 or ± 1), with maximum deviation 0.

Rotation is about the layer's own axis through the cube centre. Each section's position and orientation are updated together when a move finishes, and the next move picks its sections from those updated positions.

The defect I did find is in the twist. The shrink-fit I added last time had a 2% safety margin that applied even at zero rotation. That made the outer layer snap to 98% size the instant it started moving, and snap back to 100% when it landed. That's the pop and the not-quite-flush landing you saw.

Fixes:

- Twist: the margin now fades in with the angle, so the outer layer's scale is exactly 1.0 at 0° , 60° and 120° . It still stays inside the outline, with a worst case of 0.6px and nothing more than 2px outside.
- Shapes: they're a little smaller (circles about 82% of the cell, squares about 70%), so each floats with a clear gap and the squares no longer read as a grid of white lines. The shape pattern and colours are unchanged. I saw no seams between sections.
- Grid check: after every move, a check measures how far each section's centre and orientation is from an exact grid value, and logs an error if it exceeds $1e-6$. It's also available as `window.cubeGridDeviation()` for tests.

One layer turn, start, 50% and end: the first move of a seeded run turns the left x layer.

- Start (0.5s): one clean hexagon, grid deviation 0.
- 50% (2.0s): the layer is tilted about 45° as one rigid block, with the other 18 sections still intact. Deviation 0.
- End (3.2s): the layer's faces show new patterns, the cube is flush again as one clean hexagon, and deviation is 0.

The page logged no console errors or warnings.

✱ Brewed for 3m 50s · done 5:53 PM

› /export

└ Conversation copied to clipboard